

TCH057 – Applications mobiles et expérience usager

Projet de session

FitZone

Rapport de projet

Présenté à Lazhar Khelifi

Tyler Nichols

Michedlore Moi-Même

Session E2026

13 août 2026

1. Description de l'application

FitZone est une application mobile Android native, écrite en Java, destinée à la clientèle d'un centre d'entraînement. Elle permet à une personne inscrite de se connecter à son compte, de consulter les programmes d'entraînement auxquels elle est inscrite, de suivre les séances et leurs échéances, de soumettre un compte-rendu d'entraînement, de répondre aux quiz de connaissances, de consulter des conseils nutritionnels et de gérer son profil.

Les données partagées (comptes, programmes, séances, quiz, annonces et conseils nutritionnels) proviennent d'un serveur JSON local exposé par json-server à partir du fichier fitzone.json. Les données propres à l'utilisateur et produites par l'application (résultats de quiz, soumissions de séances, cache du profil) sont conservées dans une base SQLite locale, ce qui permet un affichage immédiat au retour sur un écran et la consultation des informations de base hors ligne.

1.1 Écrans réalisés

Écran	Rôle
Connexion	Saisie du courriel et du mot de passe, validation contre le serveur, message explicite en cas d'échec, ouverture de la session.
Inscription	Création d'un compte (prénom, nom, courriel, mot de passe, téléphone, photo en URL) avec validation des champs et envoi au serveur.
Accueil / Tableau de bord	Vue synthèse : programmes inscrits, prochaines séances, annonces récentes, nombre de quiz disponibles et résumé des statuts (soumises, validées, en retard, à faire).
Liste des programmes	Liste dynamique des programmes inscrits avec recherche par nom ou code et filtrage (tous, actifs, terminés).
Détails d'un programme	Description, coach, session, statut, annonces, et accès aux séances et aux quiz du programme.
Séances	Liste des séances du programme avec titre, échéance, statut coloré et note.
Détail d'une séance	Description, consignes, échéance, statut, note et commentaire du coach, et soumission simulée (texte ou lien).
Quiz	Liste des quiz du programme avec statut et nombre de questions ; passation d'un quiz à choix de réponses, calcul et affichage du score.
Conseils nutritionnels	Liste de dix aliments avec image, nom, courte description, calories et moment de consommation recommandé.
Profil utilisateur	Consultation et modification du prénom, du nom, du téléphone, de la photo et du mot de passe ; courriel en lecture seule ; statistiques personnelles et déconnexion.

2. Choix de conception

2.1 Organisation du code

Le code est organisé en couches, chacune isolée dans un sous-package, conformément à l'architecture demandée dans l'énoncé. L'application utilise le SDK Android, AndroidX et Material Components.

Sous-package	Contenu
activites	Les onze écrans de l'application
adaptateurs	Cinq adaptateurs RecyclerView : programmes, aperçu des séances au tableau de bord, séances, quiz et aliments
modeles	User, Program, Seance, Quiz, Question, QuizResult et Aliment
reseau	ApiClient et ApiCallback
dao	BaseSQLite et les trois DAO d'accès à la base locale
utils	SessionManager, StatutSeance, NavigationHelper et ImageLoader

Les activités ne contiennent donc aucun code réseau ni aucune requête SQL : elles appellent un DAO ou ApiClient, puis se contentent de peupler les vues. Cette séparation a permis de mettre en place les couches communes d'abord, puis de répartir les écrans entre les membres de l'équipe, chacun réutilisant les mêmes classes de base.

2.2 Couche réseau

ApiClient est une classe utilitaire statique qui expose trois méthodes : get, post et put. Chaque appel est exécuté sur un ExecutorService à un seul fil, puis la réponse est renvoyée sur le fil principal par un Handler, ce qui évite tout accès réseau sur le fil d'interface et permet aux activités de manipuler directement les vues dans le rappel. L'interface ApiCallback distingue deux issues : onSuccess avec le corps de la réponse, et onError avec un message déjà lisible par l'utilisateur (serveur injoignable ou code HTTP d'erreur).

La mise à jour du profil utilise PUT et non PATCH, car HttpURLConnection ne prend pas en charge la méthode PATCH sur Android. Comme json-server remplace intégralement l'objet visé par un PUT, l'écran Profil relit d'abord l'utilisateur complet sur le serveur, modifie les champs saisis sur cet objet frais, puis envoie le résultat. Le cache SQLite, qui ne conserve pas le mot de passe ni les inscriptions, ne sert jamais à construire le corps de la requête : sans cette précaution, un enregistrement du profil effacerait les inscriptions et empêcherait la reconnexion.

Le chargement des photos de profil distantes est assuré par ImageLoader, écrit pour l'occasion sur le même principe (exécution en arrière-plan, retour sur le fil principal). L'image de remplacement reste affichée si l'URL est invalide ou inaccessible.

2.3 Répartition des données

Chaque donnée possède une seule source de vérité, ce qui évite les incohérences entre le serveur et la base locale.

Donnée	Emplacement
Programmes, séances, quiz, annonces, conseils nutritionnels	Serveur JSON, en lecture seule
Comptes utilisateurs	Serveur JSON
Résultats de quiz	SQLite
Soumissions et statuts locaux des séances	SQLite
Cache du profil affiché hors ligne	SQLite
Identifiant de l'utilisateur connecté	SharedPreferences

La base fitzone.db est créée par BaseSQLite et compte trois tables. Les tables de résultats et de soumissions sont indexées par la paire (identifiant d'utilisateur, identifiant de l'élément), de sorte que plusieurs comptes peuvent utiliser le même appareil sans mélanger leurs données.

2.4 Navigation entre les écrans

L'écran de connexion est le point d'entrée de l'application. Après une connexion réussie, il est retiré de la pile et l'utilisateur arrive au tableau de bord.

La navigation principale repose ensuite sur une barre inférieure (BottomNavigationView) présente sur les quatre écrans de premier niveau : Accueil, Programmes, Nutrition et Profil. Elle est configurée par une classe unique, NavigationHelper, qui coche l'entrée courante et ouvre l'activité choisie en la ramenant au premier plan plutôt qu'en empilant une nouvelle instance

Les écrans de contenu (détails d'un programme, séances, détail d'une séance, liste des quiz, passation d'un quiz) s'empilent par-dessus et se referment par la flèche de retour de leur barre d'outils. Ils reçoivent leur contexte par un extra d'Intent dont la constante est déclarée dans l'activité de destination, ce qui a permis de figer le contrat entre les écrans avant même que leur contenu soit écrit :

- DetailsProgrammeActivity.EXTRA_PROGRAMME_ID
- SeancesActivity.EXTRA_PROGRAMME_ID et ListeQuizActivity.EXTRA_PROGRAMME_ID
- DetailSeanceActivity.EXTRA_SEANCE_ID et QuizActivity.EXTRA_QUIZ_ID

Ces écrans vérifient à leur ouverture la présence de l'extra attendu et, lorsqu'ils en dépendent, celle de la session. Ils se referment sinon.

2.5 Conception de l'interface et ergonomie

L'interface présente l'information sous forme de cartes et de sections, format adapté à la lecture sur téléphone. Les critères ergonomiques vus dans le cours ont guidé les décisions suivantes :

- Guidage : chaque écran porte un titre explicite, les sections sont annoncées par des intertitres et les statuts sont repris sous forme de pastilles colorées (à faire, à venir, soumise, en retard, validée) avec une couleur distincte par statut.
- Retour immédiat : les écrans de contenu affichent un indicateur de chargement pendant l'appel au serveur, les actions confirmées produisent un message court, et les listes vides affichent un texte explicatif plutôt qu'un écran blanc.
- Gestion des erreurs : les messages sont formulés en langage courant (« Courriel ou mot de passe incorrect », « Impossible de joindre le serveur ») ; un champ obligatoire laissé vide est signalé sous le champ concerné dans les formulaires de profil et de soumission.
- Homogénéité : les mêmes composants sont réutilisés d'un écran à l'autre (cartes, pastilles de statut, barre d'outils avec flèche de retour), et les textes de l'interface sont centralisés dans strings.xml, ce qui garantit un vocabulaire uniforme.
- Charge de travail : le tableau de bord se limite à trois éléments par section, les séances étant triées par échéance la plus proche, et renvoie vers la liste complète ; la recherche comme les filtres de programmes s'appliquent au fil de la saisie, sans bouton de validation.

2.6 Calcul des statuts et adaptations du fichier JSON

La règle de statut d'une séance est implémentée une seule fois, dans StatutSeance, et appelée par le tableau de bord, la liste des séances et le détail d'une séance. Les critères sont évalués dans l'ordre suivant : une note du coach donne « Validée ». Une séance dont la date de disponibilité n'est pas

atteinte est « À venir ». Une soumission enregistrée localement donne « Soumise ». Une échéance dépassée donne « En retard ». Sinon la séance est « À faire ».

Le statut « À venir » est un ajout aux quatre statuts de l'énoncé. Il évite qu'une séance pas encore ouverte apparaisse comme étant à faire, et le formulaire de soumission est masqué tant que la date de disponibilité n'est pas atteinte.

L'énoncé autorisant l'enrichissement du fichier de données, deux modifications ont été apportées à fitzone.json :

- Un champ statut (« actif » ou « termine ») sur chaque programme, sans lequel le filtrage exigé à l'écran Liste des programmes n'aurait aucune source .
- Une collection nutrition contenant les dix aliments de l'énoncé avec leur description, leurs calories et leur moment de consommation. Les photos correspondantes sont embarquées dans les ressources de l'application et associées par leur nom, afin que l'écran reste lisible sans dépendre d'un service d'images externe.

3. État du projet

Le projet est entièrement fonctionnel. Tous les écrans demandés par l'énoncé sont implémentés. Les choix suivants ont volontairement été laissés hors de la portée du projet ; ils ne correspondent pas à des fonctionnalités défailtantes, mais à des limites assumées :

- l'application ne rouvre pas automatiquement la session au lancement : même si l'identifiant de l'utilisateur est conservé, l'écran de connexion est toujours affiché en premier ;
- l'unicité du courriel à l'inscription n'est pas contrôlée, le serveur JSON acceptant deux comptes portant la même adresse ;
- les mots de passe sont comparés en clair, le serveur JSON de démonstration les stockant tels quels ;
- les soumissions de séances et les résultats de quiz sont conservés localement, comme le permet l'énoncé, et ne sont pas répliqués vers le serveur ;
- les images de programmes ne sont pas affichées : les adresses fournies dans le fichier de données pointent vers un domaine d'exemple inexistant, et l'énoncé indique cet élément comme optionnel.

La principale difficulté rencontrée en cours de développement a été le comportement de json-server, qui garde l'ensemble du fichier en mémoire et le réécrit à chaque écriture : toute modification manuelle de fitzone.json effectuée pendant que le serveur tourne est silencieusement écrasée. La règle retenue est d'arrêter le serveur avant d'éditer le fichier, puis de le relancer. La seconde difficulté concernait la mise à jour du profil, réglée par la relecture préalable.

4. Environnement de développement

Élément	Version
Langage	Java (compatibilité source et cible : Java 11)
SDK Android minimal (minSdk)	API 24 – Android 7.0
SDK Android cible (targetSdk)	API 36
SDK Android de compilation (compileSdk)	API 36.1
Gradle (wrapper)	9.4.1
Plugin Android Gradle (AGP)	9.2.1
JDK utilisé par Gradle	21
Bibliothèques	AndroidX AppCompat 1.7.1, Material Components 1.14.0, ConstraintLayout 2.2.1, Activity 1.13.0
Serveur de données	json-server

L'application déclare la permission INTERNET et autorise le trafic en clair (usesCleartextTraffic), le serveur de démonstration étant accessible en HTTP. L'adresse de base est <http://10.0.2.2:3000>, qui désigne la machine hôte depuis l'émulateur Android.

4.1 Exécution du projet

1. Lancer le serveur à la racine du dépôt : `npx json-server fitzone.json --port 3000`
2. Ouvrir le dossier code dans Android Studio, puis démarrer l'application sur un émulateur.
3. Se connecter avec un compte existant du fichier de données ou créer un compte depuis l'écran d'inscription.